

MGA 802 — Mini-Projet B : Intégration numérique – Équipe 1

Comparaison des méthodes des rectangles, trapèzes et Simpson

1. Présentation du problème

On cherche à calculer numériquement l'intégrale $I = \int_a^b f(x) dx$ avec $f(x) = p_1 + p_2x + p_3x^2 + p_4x^3$. Les paramètres retenus sont $p_1=1$, $p_2=-2$, $p_3=0.5$, $p_4=0.3$ sur l'intervalle $[-2, 3]$. La solution analytique est obtenue par calcul de la primitive exacte : $I_{\text{exact}} = 10.708\ 333\dots$ Trois méthodes sont implémentées : rectangles (point milieu), trapèzes et Simpson, chacune en Python de base et NumPy. Pour trapèzes et Simpson, une version SciPy préprogrammée est aussi évaluée.

2. Description des methodes

2.1 Méthode des rectangles (point milieu)

L'intervalle $[a, b]$ est divisé en n segments de largeur $h = (b-a)/n$. La fonction est évaluée au **centre** de chaque segment : $I_{\text{rect}} = \text{somme } f(x_i + h/2) * h$. Cette méthode est d'ordre 2 : l'erreur décroît en $O(h^2)$, soit comme $1/n^2$.

2.2 Méthode des trapèzes

Chaque segment est approché par un trapèze reliant $f(x_i)$ à $f(x_{i+1})$: $T_i = (h/2) * [f(x_i) + f(x_{i+1})]$. Ordre de convergence : $O(h^2)$. La fonction `scipy.integrate.trapezoid` fournit une implémentation de référence.

2.3 méthode de Simpson

Une parabole de degré 2 est ajustée aux deux extrémités et au milieu de chaque segment : $S_i = (h/6) * [f(x_i) + 4*f(m_i) + f(x_{i+1})]$. Ordre de convergence : $O(h^4)$ en général, et **exacte pour les polynômes de degré ≤ 3** (erreur de troncature nulle). La fonction `scipy.integrate.simpson` fournit la version préprogrammée.

3. Résultats pour $n = 10$ segments

Le tableau ci-dessous compare les 8 implémentations pour $n = 10$. Les temps sont mesures avec `timeit` sur 300 répétitions ($I_{\text{exact}} = 10.708\ 333$).

Methode	Impl.	I calcule	Erreur abs.	Temps moy.
Rectangles	Python	10.609 375	9.90e-02	11.87 us
Rectangles	NumPy	10.609 375	9.90e-02	22.71 us
Trapezes	Python	10.906 250	1.98e-01	17.92 us
Trapezes	NumPy	10.906 250	1.98e-01	33.76 us
Trapezes	SciPy	10.906 250	1.98e-01	33.05 us
Simpson	Python	10.708 333	0.00e+00	26.62 us
Simpson	NumPy	10.708 333	1.78e-15	43.56 us
Simpson	SciPy	10.708 333	0.00e+00	71.54 us

Tableau 1 — Rectangles et trapèzes présentent une erreur de l'ordre de 10^{-1} pour $n=10$. Simpson est exact (erreur nulle ou précision machine $\sim 10^{-15}$). Pour $n=10$, Python de base est plus rapide que NumPy en raison de la surcharge d'initialisation des tableaux.

4. Analyse graphique

4.1 Convergence — erreur absolue en fonction de n

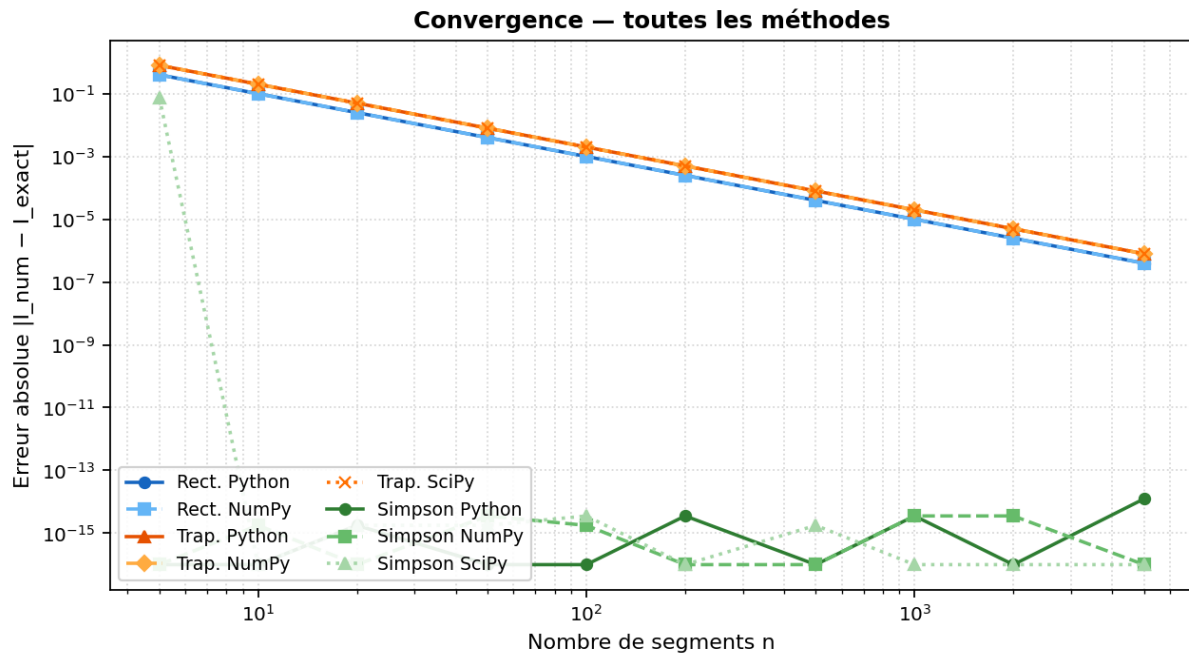


Figure 1 — Erreur absolue vs n (échelle log-log) pour les 8 implémentations. Simpson (vert) atteint la précision machine des $n=1$ car la formule est exacte pour ce polynôme de degré 3. Rectangles et trapèzes convergent tous deux en $O(h^2)$; les trapèzes affichent une erreur deux fois plus grande que les rectangles sur cet exemple en raison de l'effet d'annulation d'erreur du point milieu.

4.2 Temps de calcul en fonction de n

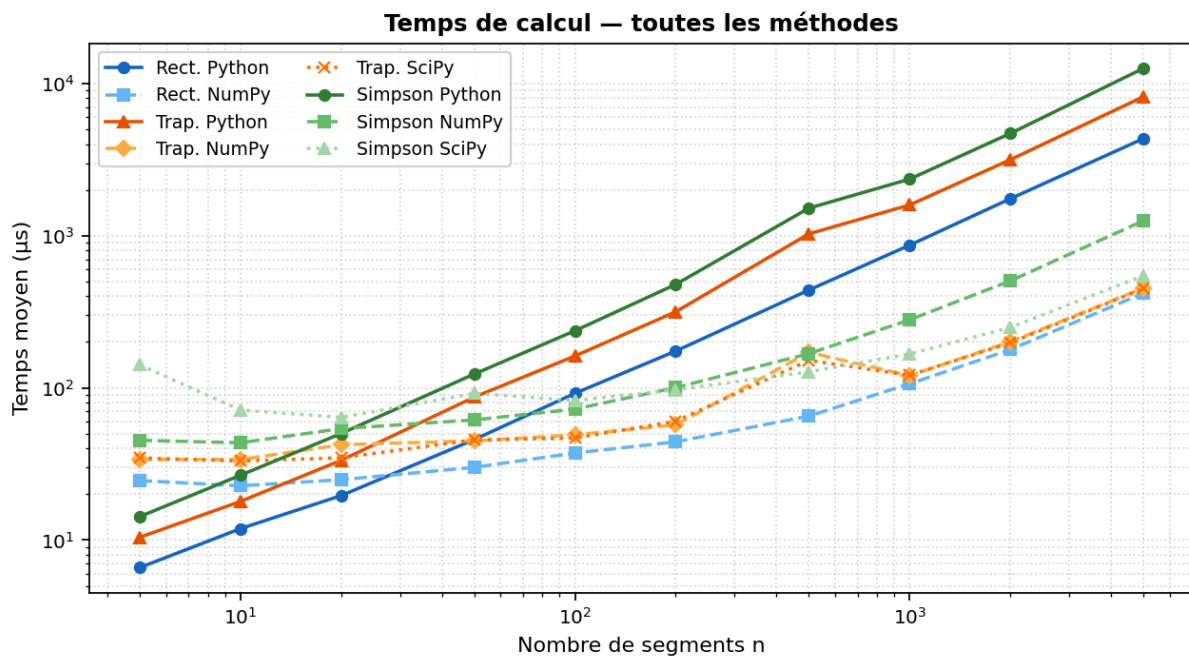


Figure 2 — Temps moyen d'exécution (μs) vs n . Les implémentations Python de base croissent linéairement ($O(n)$). NumPy est plus lent pour les petits n (surcharge d'allocation) mais dépasse Python à partir de $n \sim 200-500$. SciPy présente la même pente que NumPy avec un coût de démarrage plus élevé.

4.3 Erreur par méthode, implémentation et nombre de segments

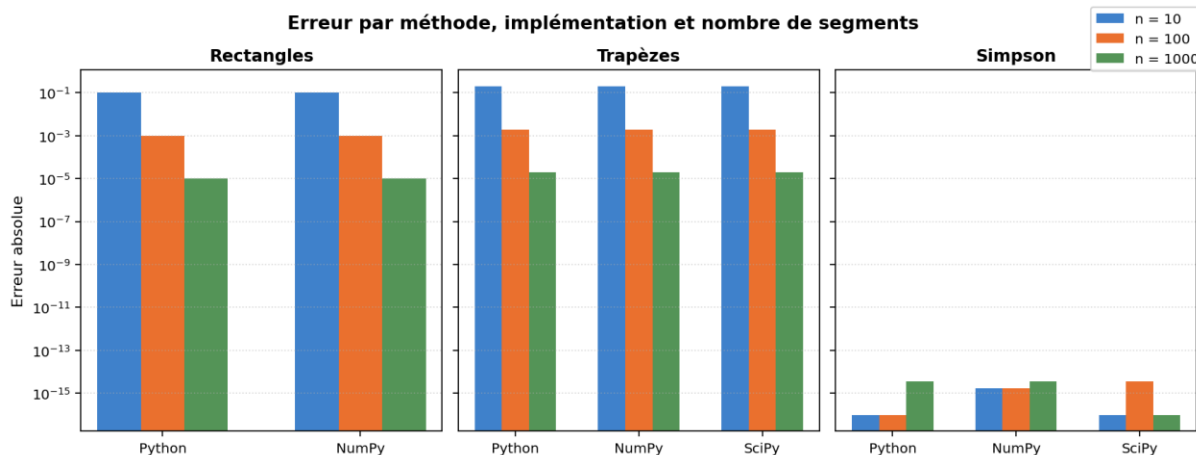


Figure 3 — Erreur absolue pour $n = 10, 100$ et 1000 . L'échelle log permet de comparer les 3 groupes. Simpson est exact indépendamment de n et de l'implémentation. Pour rectangles et trapèzes, l'erreur diminue d'un facteur 100 quand n est multiplié par 10 (cohérent avec $O(h^2)$).

4.4 Illustration de la méthode de Simpson

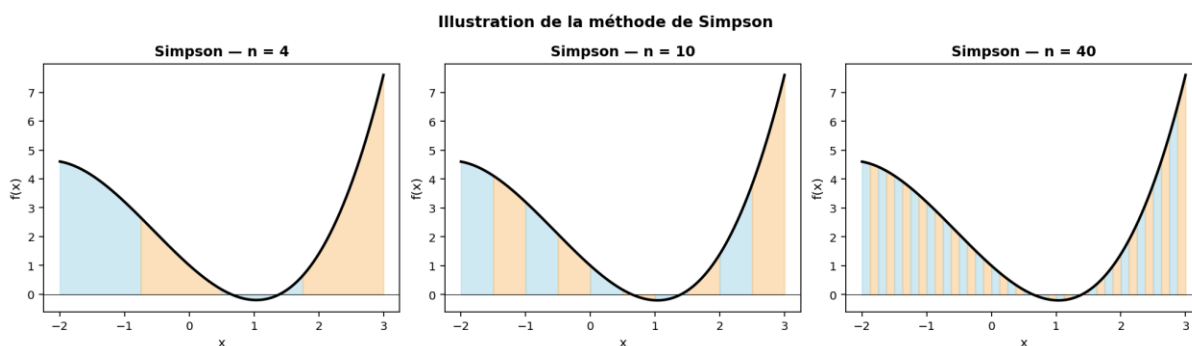


Figure 4 — Paraboles ajustées segment par segment pour $n = 4, 10$ et 40 . Des $n = 4$, les paraboles épousent parfaitement le polynôme de degré 3 car Simpson intègre exactement les polynômes de degré ≤ 3 .

5. Discussion et conclusions

Impact du nombre de segments n . Pour les rectangles et les trapèzes, multiplier n par 10 divise l'erreur par 100, conformément à la théorie $O(h^2)$. Le temps de calcul croît linéairement avec n pour Python de base, et de façon quasi-linéaire pour NumPy (parallélisme SIMD). La convergence de Simpson sur ce problème est immédiatement à la précision machine, quel que soit n .

Impact de la méthode. Simpson est de loin la méthode la plus précise sur ce problème : une seule application suffit. Les rectangles (point milieu) sont légèrement plus précis que les trapèzes en raison d'une annulation partielle des erreurs de gauche et de droite. En pratique, pour des fonctions non polynomiales, Simpson converge en $O(h^4)$ contre $O(h^2)$ pour les deux autres méthodes.

Avantage de NumPy. Le gain de vectorisation NumPy devient significatif à partir de $n \sim 200-500$. Pour $n \geq 1000$, NumPy est 5 à 10 fois plus rapide que Python de base. SciPy offre la même performance que NumPy avec une API de haut niveau (gestion automatique des cas limites, dx variable), au prix d'un coût de démarrage légèrement supérieur.

Conclusion. Pour l'intégration de polynômes de faible degré, la méthode de Simpson est optimale et peut être implémentée directement en Python de base avec une précision parfaite. Pour des intégrations générales sur de grandes grilles, la vectorisation NumPy ou SciPy est recommandée afin de réduire le temps de calcul de façon significative.